Université des sciences et de la technologie d'Oran Mohamed Boudiaf – USTO MB
Faculté des Mathématiques et de l'Informatique - Département d'informatique
1 ier année Master – S1 - 2026 / 2027

Méthodes d'optimisation

Chapitre 1 : Fondamentaux de l'optimisation

Cours 2
Partie 2: Optimisation sans contraintes

Présenté Par : Dr. BEKKOUCHE .I

Plan cours 2
Chapitre 1 partie 2 : Optimisation sans contraintes

- Recherche linéaire (line search).
- Méthodes de descente : Gradient Descent, Newton, Quasi-Newton (BFGS).
- Taux de convergence.

Optimisation sans contraintes

Introduction

Introduction

Soit f (**X**) une fonction de $\Re^n \rightarrow \Re^n$; Le problème d'optimisation sans contraintes s'écrit de la façon suivante :

On veut résoudre $\boldsymbol{min_{x \in S \subset Rn} f(x)}$ ou $\boldsymbol{max_{x \in S \subset Rn} f(x)}$ i.e. on cherche $\boldsymbol{v}$ valeur optimale et $\boldsymbol{x_0}$ tel que $\boldsymbol{f(x_0)=v.}$

La fonction f (**X**) est souvent appelée :

- fonction coût,
- fonction objectif,
- critère d'optimisation.

Introduction

Soit $x_0 \in S$.

1. On dit que $\boldsymbol{x_0}$ est un **minimum local** si: $\boldsymbol{\exists V \in V(x_0) \text{ tel que } \forall x \in V, f(x) \geq f(x_0)}$
2. On dit que $\boldsymbol{x_0}$ est un **minimum local strict** si: $\boldsymbol{\exists V \in V(x_0) \text{ tel que } \forall x \in V, f(x) > f(x_0)}$
3. On dit que $\boldsymbol{x_0}$ est un **minimum global** si: $\boldsymbol{\forall x \in S, f(x) \geq f(x_0)}$
4. On dit que $\boldsymbol{x_0}$ est un **minimum global strict** si: $\boldsymbol{\forall x \in S, f(x) > f(x_0)}$

On definit de la même façon un ***maximum local***, ***maximum local strict***, ***maximum global***, ***maximum global strict***, en renversant les inégalités.

Propriétés

1) Si x_0 est un minimum global, alors c'est un minimum local.

2) Si on connait tous les minima locaux alors le plus petit est le minimum global.

Introduction

Tous les algorithmes présentés suivent le même schéma itératif général :

$\mathbf{x_{k+1} = x_k + \alpha_k p_k}$

où $\mathbf{p_k}$ est une **direction de descente** ➔ vérifiant $\mathbf{\nabla f(x_k)^T p_k < 0}$, c.-à-d. que **f** décroît localement le long de $\mathbf{p_k}$

et $\mathbf{\alpha_k > 0}$ est un **pas** (step length) déterminé par une procédure de recherche linéaire.

x ← point de départ

répéter :

1. choisir une direction **p** *(dans quel sens aller ?)*
2. 2. choisir un pas α > 0 *(de combien avancer ?)*
3. 3. **x** ← **x + α · p** (avancer) jusqu'à ce que **‖∇f(x)‖ < ε** *(terrain presque plat : on s'arrête)*

Rappels

➢ **Dérivée f'(x)** : la ***pente*** de la courbe au point ***x***.

Si **f' > 0** la **courbe monte**, si **f' < 0** elle **descend**, si **f' = 0** elle est **plate** (sommet ou fond).

La dérivée en x_0 est définie par la limite :

$$f'(x_0)=\lim_{h\to 0} (f(x_0+h)-f(x_0))/ h$$

- Le numérateur $\mathbf{f(x_0+h)-f(x_0)}$ est la variation de la fonction.
- Le dénominateur **h** est la variation de la variable.
- Le rapport est la pente moyenne, et la limite quand $\mathbf{h\to 0}$ donne la pente instantanée.

Rappels

➢ Dérivée f’(x) :

Exemple : $f(x) = x^2$,

$f'(x)=\lim_{h\to 0}((x+h)^2-x^2)/h = \lim_{h\to 0}(x^2+2xh+h^2-x^2)/h = \lim_{h\to 0}(2xh+h^2)/h = \mathbf{\lim_{h\to 0} 2x+h}$

Quand h→0, il reste : **f’(x)=2x ➔** La dérivée f’(x) donne la **pente de la courbe** en chaque point x

f’(x) = 2x ; en **x = 3** la pente vaut **6** (ça monte vite), en **x = 0** elle vaut **0** (c'est le fond).

- La fonction **$f(x)=x^2$** est une **parabole** ouverte vers le haut.
- La dérivée **f’(x)=2x** indique :
- **À gauche de 0 (x<0)**, la pente est négative → la courbe descend.
- **À droite de 0 (x>0)**, la pente est positive → la courbe monte.
- En **x=0**, la pente est nulle → point critique, **minimum global.**

Rappels

➢ **Dérivée f’(x)** :

Règles de dérivation :

Au lieu de refaire la limite à chaque fois, on utilise des **règles de dérivation** :

- $(x^n)' = nx^{n-1}$
- $(af(x))' = af'(x)$
- $(f(x)+g(x))' = f'(x)+g'(x)$
- $(f(x)g(x))' = f'(x)g(x)+f(x)g'(x)$

Rappels

- **Gradient ∇f :** la **dérivée** quand il y a **plusieurs variables**, c'est-à-dire un vecteur contenant une pente par direction.
- Le gradient pointe vers la **montée la plus forte**.
- Le vecteur −∇f pointe vers la **descente la plus forte** (utilisé en optimisation).

Si **f(x, y)** est une fonction de deux variables

alors son gradient est :

∇f(x, y)=(∂f/∂x, ∂f/∂y)

$$\nabla f = \frac{\partial f}{\partial \mathbf{X}} = \left[\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_n} \right]$$

•∂f/∂x = pente quand on bouge seulement x.

•∂f/∂y = pente quand on bouge seulement y.

Rappels

➢ Gradient ∇f :

Exemple : $f(x, y) = x^2 + y^2$, $\nabla f = (2x, 2y)$; en (1, 2), $\nabla f = (2, 4)$.

•Dérivée partielle par rapport à x : $\partial f/\partial x = 2x$

•Dérivée partielle par rapport à y : $\partial f/\partial y = 2y$

Donc : $\nabla f(x,y) = (2x, 2y)$

Application en (1,2) : $\nabla f(1,2) = (2 \cdot 1,\ 2 \cdot 2) = (2,4)$ ➔ indique la direction où la fonction f(x,y) augmente le plus vite.

Pour descendre vers le minimum, on prend la direction opposée : $-\nabla f = (-2,-4)$.

Rappels

- **Hessienne $\nabla^2 f$:** la ***courbure***, c'est-à-dire la vitesse à laquelle la pente change.

- En une variable, c'est f″(x).

•Si f″(x)>0 → la fonction est en forme de **U** (minimum, “bol”).

•Si f″(x)<0 → la fonction est en forme de ∩ (maximum, “dôme”).

- En **plusieurs variables** : la Hessienne est une **matrice** qui contient toutes les dérivées secondes (pentes qui changent dans chaque direction).

Pour une fonction f(x,y), la Hessienne est :

$\nabla 2f(x,y)=[\partial^2 f/\partial x^2, \partial^2 f/\partial x\partial y ; \partial^2 f/\partial y\partial x, \partial^2 f/\partial y^2]$

$$\mathbf{H}_f = \frac{\partial^2 f(\mathbf{X})}{(\partial \mathbf{X})^2} = \begin{bmatrix} \frac{\partial^2 f(\mathbf{X})}{(\partial x_1)^2} & \frac{\partial^2 f(\mathbf{X})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{X})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(\mathbf{X})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{X})}{(\partial x_2)^2} & \cdots & \frac{\partial^2 f(\mathbf{X})}{\partial x_2 \partial x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial^2 f(\mathbf{X})}{\partial x_n \partial x_1} & \frac{\partial^2 f(\mathbf{X})}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{X})}{(\partial x_n)^2} \end{bmatrix}$$

Rappels

➢ Hessienne $\nabla^2 f$:

Exemple fonction d'une variable: $f(x) = x^2$

Dérivée première

$f'(x)=2x$ Cela donne la pente.

Dérivée seconde

$f''(x)=2$ La dérivée seconde est **constante et positive**.

•Cela signifie que la courbe est toujours en forme de **U** (parabole ouverte vers le haut).

•Le minimum est en $x=0$.

Rappels

➢ Hessienne $\nabla^2 f$:

Exemple fonction de deux variables : $f(x,y)=x^2+y^2$

Dérivées partielles premières: $\partial f/\partial x=2x$, $\partial f/\partial y=2y$ ➔ C'est le **gradient** : $\nabla f(x,y)=(2x,2y)$.

dérivées partielles secondes: $\partial^2 f/\partial x^2=2$, $\partial^2 f/\partial y^2=2$, $\partial^2 f/\partial x\partial y=0$, $\partial^2 f/\partial y\partial x=0$

Hessienne : $\nabla^2 f(x,y)=[2\ 0\ ,\ 0\ 2]$ ➔ C'est une matrice diagonale avec des valeurs positives

- Les valeurs propres de la Hessienne sont 2 et 2.
- Comme elles sont toutes **positives**, la surface est en forme de **bol** (paraboloïde).
- Le minimum est en (0,0).

Rappels

- **Direction de descente** : **p** est une direction de descente si $\nabla f(x)^T p < 0$ (ce produit scalaire négatif veut dire qu'on va globalement à l'opposé du gradient, donc **f** diminue au début du déplacement).
- **Itération** : on répète la même recette en repartant du dernier résultat obtenu.

Exemple

$f(x, y) = x^2 + 4y^2$ point de départ **$x_0 = (4, 1)$**

- Le minimum est évident : **$x^2 \geq 0$** et **$4y^2 \geq 0$**, donc le minimum est en **(0, 0)** avec **$f = 0$.**
- Au départ : $f(x_0) = 4^2 + 4 \cdot 1^2 = 16 + 4 =$ **20**.
- *Gradient* : $\partial f/\partial x = 2x$ (le terme $4y^2$ est une constante pour x, sa dérivée est 0) et $\partial f/\partial y = 8y$.

Donc **$\nabla f = (2x, 8y)$**. En **x_0 : $\nabla f(x_0) = (8, 8)$.**

- *Hessienne* : on dérive encore. **$\partial^2 f/\partial x^2 = 2$, $\partial^2 f/\partial y^2 = 8$**, et les dérivées croisées valent 0.

Donc **$H = [[2, 0], [0, 8]]$**, constante partout.

- Interprétation : *la courbure vaut 2 selon x et 8 selon y. La « vallée » est 4 fois plus raide selon y que selon x. Le rapport **$\kappa = 8/2 = 4$** s'appelle le **conditionnement**.*

Optimisation sans contraintes

Recherche linéaire (line search)

Recherche linéaire (line search)

Principe :

Dans un problème d'optimisation, on veut **minimiser une fonction** f(x).

- On connaît la **direction de descente** (souvent donnée par le gradient négatif **$-\nabla f$**).
- Mais il reste à décider **quelle longueur de pas α** on doit prendre dans cette direction.

Donc la **recherche linéaire** sert à trouver ce pas **α.**

Recherche linéaire (line search)

Principe :

- Une fois la direction **p** choisie, il faut décider de combien avancer.
- Un **pas trop petit** fait perdre du temps
- un **pas trop grand** fait *dépasser* le fond de la vallée et **f** remonte *(comme freiner trop tard en voiture).*
- On cherche donc un pas **α** qui fait baisser **f** « suffisamment », sans chercher le pas parfait *(trop coûteux).*

Recherche linéaire (line search)

Exemple intuitif :

Imagine que tu descendes une montagne :

- Tu sais dans quelle direction aller (vers le bas).
- La recherche linéaire te dit **quelle longueur de pas faire** :
  - Petit pas → tu avances lentement.
  - Trop grand pas → tu risques de remonter une autre pente.
  - Pas “juste” → tu descends efficacement.

Recherche linéaire (line search)

Applications

- **Machine learning** : ajuster les paramètres des modèles.
- **Ingénierie** : optimisation de structures.
- **Finance** : minimisation de fonctions de risque.

Recherche linéaire (line search)

Définition mathématique :

Si on est en un **point x_k** et qu'on a une **direction d_k** (gradient négatif **$-\nabla f$**), alors :

$$x_{k+1}=x_k+\alpha d_k$$

La recherche linéaire consiste à choisir **α** qui réduit le plus la fonction f(x).

Recherche linéaire (line search)

Exemple simple en 1D: $f(x)=(x-2)^2$

- Minimum en x=2.
- Point de départ : $x_0=0$.
- Direction de descente : vers la droite (d=1).

On teste différents pas α :

- $\alpha=1 \Rightarrow f(1)=1$
- $\alpha=2 \Rightarrow f(2)=0$ minimum atteint
- $\alpha=3 \Rightarrow f(3)=1$ (on est remonté)

La recherche linéaire choisit **α=2**.

Exemple en 2D: $f(x,y)=x^2+y^2$

$$x_{k+1}=x_k+\alpha d_k$$

- Minimum en (0,0).
- Point de départ : (1,2).
- Gradient : $\nabla f(1,2)=(2,4)$.
- Direction de descente : $-\nabla f=(-2,-4)$.

On cherche α tel que : $(x,y)_{new}=(1,2)+\alpha(-2,-4)=(1-2\alpha,\ 2-4\alpha)$

La fonction devient :

$f(x,y)=(1-2\alpha)2+(2-4\alpha)2$

On choisit α qui minimise cette expression.

Ici, le calcul donne **α=0.5**, ce qui amène au minimum (0,0).

Recherche linéaire (line search)

1. Recherche linéaire exacte

On cherche le **pas optimal** α en résolvant le problème **à une dimension** : $\mathbf{\min_{\alpha>0} f(x+\alpha d)}$

Exemple

Soit : $f(x)=(x-2)^2, x_0=0, d=1$

On veut minimiser : $f(0+\alpha\cdot 1)=(\alpha-2)^2$

- Développons : $f(\alpha)=\alpha^2-4\alpha+4$
- Dérivée : $f'(\alpha)=2\alpha-4$
- On résout $f'(\alpha)=0\Rightarrow\alpha=2$

Pas optimal exact : $\boldsymbol{\alpha=2}$. On atteint directement le minimum x=2.

Recherche linéaire (line search)

2. Recherche linéaire inexacte

On ne cherche pas le pas parfait, mais un pas **suffisant** qui réduit bien la fonction.

- Plus rapide, utilisé en pratique.
- On applique des **conditions** (Armijo, Wolfe) qui garantissent une amélioration sans calculer exactement le minimum.

Exemple

Même fonction : $f(x)=(x-2)^2, x_0=0$, $d=1$

On teste des pas α :

- $\alpha=1 \Rightarrow f(1)=1$ (amélioration par rapport à $f(0)=4$)
- $\alpha=2 \Rightarrow f(2)=0$ (minimum exact)
- $\alpha=1.5 \Rightarrow f(1.5)=0.25$ (bonne amélioration)

En recherche inexacte, on pourrait s'arrêter à **$\alpha=1.5$** car la fonction a bien diminué, même si ce n'est pas le minimum exact.

Recherche linéaire (line search)

Condition de décroissance suffisante (condition d'Armijo) :

La pente nous « promet » une baisse de **f** de $\alpha \cdot |\nabla f(x)^T p|$ pour un pas α.

On exige que la baisse **réelle** soit au moins une petite fraction c_1 de cette baisse promise :

$$f(x + \alpha \cdot p) \leq f(x) + c_1 \cdot \alpha \cdot (\nabla f(x)^T p)$$

le terme $\nabla f(x)^T p$ est négatif, donc le membre de droite est inférieur à **f(x).**

En pratique $c_1 = 10^{-4}$; dans nos exemples on prend $c_1 = 0{,}1$ pour que les chiffres soient lisibles.

Recherche linéaire (line search)

Algorithme (backtracking = « reculer jusqu'à ce que ça marche »):

$\alpha \leftarrow 1$

tant que $f(x + \alpha \cdot p) > f(x) + c_1 \cdot \alpha \cdot (\nabla f(x)^T p)$:

$\alpha \leftarrow \alpha / 2$ # le pas est trop grand : on le réduit

retourner α

Recherche linéaire (line search)

Exemple: $f(x, y) = x^2 + 4y^2$ au point $x_0 = (4, 1)$, on prend la direction du gradient : $p = -\nabla f(x_0) = (-8, -8)$.

- Baisse « promise » : $\nabla f(x_0)^T p = 8 \cdot (-8) + 8 \cdot (-8) = -128$.
- Seuil d'acceptation avec $c_1 = 0{,}1$: $f(x_0) + 0{,}1 \cdot \alpha \cdot (-128) = 20 - 12{,}8 \cdot \alpha$.

α testé	Nouveau point $x_0 + \alpha \cdot p$	f(nouveau point)	Seuil $20 - 12{,}8 \cdot \alpha$	Verdict
1	$(4-8, 1-8) = (-4, -7)$	$16 + 4 \cdot 49 = 212$	7,2	$212 > 7{,}2$: **refusé** (on a dépassé la vallée)
0,5	$(4-4, 1-4) = (0, -3)$	$0 + 4 \cdot 9 = 36$	13,6	$36 > 13{,}6$: **refusé**
0,25	$(4-2, 1-2) = (2, -1)$	$4 + 4 \cdot 1 = 8$	16,8	$8 \leq 16{,}8$: **accepté**

On retient $\alpha = 0{,}25$: f passe de 20 à 8.

$$\mathbf{f(x + \alpha \cdot p) > f(x) + c_1 \cdot \alpha \cdot (\nabla f(x)^T p)}$$

Recherche linéaire (line search)

Comparaison

Méthode	Principe	Avantage	Inconvénient
Exacte	Trouver le α qui minimise exactement f(x+αd)	Optimal, précis	Peut être coûteux (nécessite dérivées, calculs)
Inexacte	Trouver un α qui réduit suffisamment f	Rapide, pratique	Pas toujours optimal

Optimisation sans contraintes

Méthodes de descente :

Gradient Descent

λ

Méthodes de descente : *Gradient Descent*

Principe

À chaque étape, on regarde la pente sous nos pieds et on fait un pas dans la direction où ça descend le plus vite, c'est-à-dire l'opposé du gradient.

C'est la méthode la plus simple : elle n'a besoin que du gradient.
x_{k+1} = x_k – α · ∇f(x_k)

λ

Méthodes de descente : *Gradient Descent*

Algorithme

$x \leftarrow x_0$
pour k = 0, 1, 2, ... :
$g \leftarrow \nabla f(x)$
si $\|g\| < \varepsilon$: arrêter
$p \leftarrow -g$
choisir α (pas fixe, ou par backtracking)
$x \leftarrow x + \alpha \cdot p$

λ

Méthodes de descente : *Gradient Descent*

Exemple 1: $f(x) = x^2$, $x_0 = 4$. Ici $f'(x) = 2x$, donc la règle devient **$x_{k+1} = x_k - \alpha \cdot 2x_k = (1 - 2\alpha) \cdot x_k$** : à chaque itération, x est multiplié par $(1 - 2\alpha)$.

Pas α	Facteur (1−2α)	Valeurs successives de x	Résultat
0,25	0,5	4 → 2 → 1 → 0,5 → …	converge vers 0
0,5	0	4 → 0	converge en 1 itération (pas idéal)
1,1	−1,2	4 → −4,8 → 5,76 → −6,912 → …	**diverge** (pas trop grand)

λ

Méthodes de descente : *Gradient Descent*

Exemple 2: $f(x, y) = x^2 + 4y^2$, $x_0 = 4$. avec le pas $\alpha = 0{,}2$. On rappelle $\nabla f(x, y) = (2x, 8y)$.

Itération 0. $x_0 = (4, 1)$, $f = 20$, $\nabla f = (8, 8)$. $x_1 = x_0 - 0{,}2 \cdot \nabla f = (4 - 0{,}2 \cdot 8, 1 - 0{,}2 \cdot 8) = (4 - 1{,}6, 1 - 1{,}6) =$ **(2,4 ; –0,6)**, et $f(x_1) = 2{,}4^2 + 4 \cdot 0{,}6^2 = 5{,}76 + 1{,}44 =$ **7,2**.

Itération 1. $\nabla f(x_1) = (2 \cdot 2{,}4 \ ; \ 8 \cdot (-0{,}6)) = (4{,}8 \ ; \ -4{,}8)$. $x_2 = (2{,}4 - 0{,}2 \cdot 4{,}8 \ ; \ -0{,}6 - 0{,}2 \cdot (-4{,}8)) = (2{,}4 - 0{,}96 \ ; \ -0{,}6 + 0{,}96) =$ **(1,44 ; 0,36)**, et $f(x_2) = 2{,}0736 + 4 \cdot 0{,}1296 = 2{,}0736 + 0{,}5184 =$ **2,592**.

Itération 2. $\nabla f(x_2) = (2{,}88 \ ; \ 2{,}88)$. $x_3 = (1{,}44 - 0{,}576 \ ; \ 0{,}36 - 0{,}576) =$ **(0,864 ; –0,216)**, et $f(x_3) = 0{,}7465 + 0{,}1866 =$ **0,9331**.

Pourquoi ces chiffres ? Pour la première coordonnée, x_{k+1} = x_k − 0,2·2x_k = 0,6·x_k. Pour la seconde, y_{k+1} = y_k − 0,2·8y_k = −0,6·y_k. Donc $x_k = 4 \cdot 0{,}6^k$ et $y_k = (-0{,}6)^k$. La coordonnée y **change de signe à chaque itération** : c'est le **zigzag** (on rebondit d'un côté à l'autre de la vallée). Pour avoir $\|x\| < 10^{-6}$, il faut environ **30 itérations**.

λ

Méthodes de descente : *Gradient Descent*

Exemple 2: $f(x, y) = x^2 + 4y^2$ et puisque **$(f(x)+g(x))'=f'(x)+g'(x)$** donc

$f(x) = x^2$, Ici $f'(x) = 2x$, donc la règle devient **$x_{k+1} = x_k - \alpha \cdot 2x_k = (1 - 2\alpha) \cdot x_k$**

$f(y) = y^2$, Ici $f'(y) = 8y$, donc la règle devient **$y_{k+1} = y_k - \alpha \cdot 8y_k = (1 - 8\alpha) \cdot y_k$**

Avec un pas α, x est multiplié par (1 − 2α) et y par (1 − 8α) à chaque itération.

α	facteur sur x	facteur sur y	Comportement
0,1	0,8	0,2	converge, mais lentement (x décroît en $0,8^k$)
0,2	0,6	−0,6	**meilleur pas** : converge le plus vite
0,25	0,5	−1	y oscille entre 1 et −1 : **ne converge pas**
0,3	0,4	−1,4	**diverge** (y grandit)

λ

Méthodes de descente : *Gradient Descent*

Règle pratique :

pour converger il faut $\alpha < 2/\lambda_max = 2/8 = 0,25$, et le meilleur pas fixe est $2/(\lambda_min + \lambda_max) = 2/10 = 0,2$ (λ_min et λ_max sont les plus petite et plus grande courbures, ici 2 et 8).

Points faibles.

1. le choix du pas α (d'où la recherche linéaire) ;
2. le zigzag : plus la vallée est allongée (κ grand), plus la méthode est lente.

Optimisation sans contraintes

Méthodes de descente :

Méthode de Newton

λ

Méthodes de descente : *Méthode de Newton*

Principe

La descente de gradient ne regarde que la pente. Newton regarde aussi la **courbure** : à chaque étape, il remplace la fonction par la **parabole** (ou le « bol » en dimension **n**) qui lui ressemble localement, puis **saute directement au fond de ce bol**. Comme on connaît la forme du bol, on sait de combien avancer : plus besoin de choisir le pas.

En une variable : **$x_{k+1} = x_k - f'(x_k) / f''(x_k)$.**

En n variables : ***$x_{k+1} = x_k - H^{-1} \nabla f(x_k)$.***

En pratique, on ne calcule pas l'inverse de H : on **résout le système linéaire $H \cdot p = -\nabla f$**, puis ***$x_{k+1} = x_k + p$.***

λ

Méthodes de descente : *Méthode de Newton*

Algorithme

$x \leftarrow x_0$
pour k = 0, 1, 2, ... :
$g \leftarrow \nabla f(x)$, $H \leftarrow \nabla^2 f(x)$
si $\|g\| < \varepsilon$: arrêter
résoudre $H \cdot p = -g$ # p = direction de Newton
$x \leftarrow x + p$ # (ou $x + \alpha \cdot p$ avec backtracking, $\alpha_0 = 1$)

λ

Méthodes de descente : *Méthode de Newton*

Exemple 1

une variable, fonction qui n'est pas une parabole : $f(x) = e^x - 2x$.

On a $f'(x) = e^x - 2$ et $f''(x) = e^x$. Le minimum exact est là où $f' = 0$, donc $e^x = 2$, soit $x^* = \ln 2 \approx 0{,}693147$. On part de $x_0 = 0$.

k	x_k	f'(x_k)	f''(x_k)	Pas −f'/f''	x_{k+1}	Erreur |x_{k+1} − x*|
0	0	−1	1	+1	1	0,307
1	1	0,718	2,718	−0,264	0,7358	0,0426
2	0,7358	0,0871	2,0871	−0,0417	0,69404	0,0009
3	0,69404	0,00179	2,0018	−0,00090	0,693147	$\approx 4 \cdot 10^{-7}$

Observez les erreurs : 0,307 → 0,0426 → 0,0009 → 0,0000004. Le nombre de décimales exactes **double** à peu près à chaque itération (convergence quadratique).

λ

Méthodes de descente : *Méthode de Newton*

Exemple 2: **Hessienne diagonale**

$f(x, y) = x^2 + 4y^2$, En $x_0 = (4, 1)$, $\nabla f = (8, 8)$ et $H = [[2,0],[0,8]]$.

Résoudre $H \cdot p = -g$ donne, ligne par ligne : $2 \cdot p_1 = -8$ donc $p_1 = -4$, et $8 \cdot p_2 = -8$ donc $p_2 = -1$.

Ainsi $p = (-4, -1)$ et $x_1 = (4, 1) + (-4, -1) =$ **$(0, 0)$**, c'est-à-dire le minimum exact en **une seule itération** (contre ≈ 30 pour la descente de gradient).

λ

Méthodes de descente : *Méthode de Newton*

Exemple 2: Hessienne non diagonale : $f(x, y) = x^2 + xy + 2y^2$, $x_0 = (1, 1)$.

- **Gradient :** $\partial f/\partial x = 2x + y$ et $\partial f/\partial y = x + 4y$. En (1, 1) : $g = (3, 5)$.
- **Hessienne :** $H = [[2, 1], [1, 4]]$ (constante).
- On résout $H \cdot p = -g$, soit le système :
  - $2 \cdot p_1 + p_2 = -3$
  - $p_1 + 4 \cdot p_2 = -5$
- **De la première équation :** $p_2 = -3 - 2 \cdot p_1$. On remplace dans la seconde : $p_1 + 4(-3 - 2 \cdot p_1) = -5$, donc $p_1 - 12 - 8 \cdot p_1 = -5$, donc $-7 \cdot p_1 = 7$, donc $p_1 = -1$, puis $p_2 = -3 + 2 = -1$.
- *Vérification* : $2(-1) + (-1) = -3$ ✓ et $(-1) + 4(-1) = -5$ ✓.
- $x_1 = (1, 1) + (-1, -1) =$ **(0, 0)**, qui est bien le minimum ($f = (x + y/2)^2 + (7/4)y^2 \geq 0$, nul en (0,0)).

λ

Méthodes de descente : *Méthode de Newton*

Règle à retenir : sur une fonction **quadratique**, Newton trouve le minimum **en une seule itération**, quel que soit le point de départ.

Points faibles.

•Il faut calculer la Hessienne (n^2 dérivées secondes) et résoudre un système n×n : coût de l'ordre de n^3 opérations par itération.

•Newton cherche un point où le gradient est nul, **pas forcément un minimum**.

Exemple : $f(x) = -x^2$ (un sommet). Alors $f' = -2x$, $f'' = -2$, et $x_1 = x_0 - (-2x_0)/(-2) = x_0 - x_0 = 0$: Newton saute directement sur le **maximum**. Cela arrive quand H n'est pas définie positive (courbure négative).

•Loin du minimum, le pas peut être mauvais : on le protège avec un backtracking ($\alpha_0 = 1$) et, si besoin, on corrige H (par exemple en ajoutant un multiple de l'identité pour la rendre définie positive).

Optimisation sans contraintes

Méthodes de descente :

Méthode quasi-Newton : BFGS

λ

Méthodes de descente : *Méthode quasi-Newton : BFGS*

Principe

- Newton est très rapide mais cher : il faut calculer H à chaque itération.
- BFGS (Broyden, Fletcher, Goldfarb, Shanno) a une idée astucieuse : **ne pas calculer H, mais la deviner** au fil des itérations en observant comment le gradient change quand on se déplace.
- C'est comme apprendre la forme d'une colline en mesurant la pente à chaque endroit où l'on est passé.

On maintient une matrice **H** (qui approche l'*inverse* de la Hessienne). On la démarre à l'identité, $H_0 = I$: la première itération est donc une simple descente de gradient. À chaque itération, on observe deux vecteurs :

- **s = x_nouveau − x_ancien** (de combien on s'est déplacé) ;
- **y = ∇f(x_nouveau) − ∇f(x_ancien)** (de combien le gradient a changé).

λ

Méthodes de descente : *Méthode quasi-Newton : BFGS*

Principe

Intuition (en une variable) : si en bougeant de **s** le gradient a changé de **y**, la courbure est d'environ **y/s**, donc l'inverse de la courbure est **s/y**.
En dimension **n**, la formule de mise à jour généralise cette idée :

$$\rho = 1 / (y^T s)$$

$$H_nouveau = (I - \rho \cdot s \cdot y^T) \cdot H \cdot (I - \rho \cdot y \cdot s^T) + \rho \cdot s \cdot s^T$$

Cette mise à jour garantit l'**équation sécante $H_nouveau \cdot y = s$** (la nouvelle approximation « explique » ce qu'on vient d'observer),
et **H** reste symétrique et définie positive tant que **$y^T s > 0$** (c'est ce que garantissent les conditions de Wolfe).

λ

Méthodes de descente : *Méthode quasi-Newton : BFGS*

Algorithme

$x \leftarrow x_0$; $H \leftarrow I$; $g \leftarrow \nabla f(x)$
pour k = 0, 1, 2, ... :
si $\|g\| < \varepsilon$: arrêter
$p \leftarrow -H \cdot g$ # direction (comme Newton, mais avec H approchée)
$\alpha \leftarrow$ recherche linéaire # (Wolfe, ou backtracking)
$x_new \leftarrow x + \alpha \cdot p$
$g_new \leftarrow \nabla f(x_new)$
$s \leftarrow x_new - x$; $y \leftarrow g_new - g$
si $y^T s > 0$: mettre à jour H avec la formule BFGS
$x \leftarrow x_new$; $g \leftarrow g_new$

λ

Méthodes de descente : *Méthode quasi-Newton : BFGS*

Exemple: **$f(x, y) = x^2 + xy + 2y^2$,** Pour avoir des chiffres propres, on utilise ici le pas exact $\alpha^* = -(g^T p)/(p^T H p)$, la vraie Hessienne diag(2, 8) n'est utilisée que pour ce calcul du pas, jamais pour la direction.

Itération 1 ($H_0 = I$):

1.$x_0 = (4, 1)$, $g_0 = (8, 8)$.

2.Direction : $p_0 = -H_0 \cdot g_0 =$ **$(-8, -8)$** (c'est la descente de gradient **:** $\partial f/\partial x = 2x + y$ et $\partial f/\partial y = x + 4y$).

3.Pas exact : $\alpha = 0{,}2$ (calculé plus haut). Donc $x_1 = x_0 + \alpha \cdot p_0 = (4 - 1{,}6 ; 1 - 1{,}6) =$ **$(2{,}4 ; -0{,}6)$** et $g_1 =$ **$(4{,}8 ; -4{,}8)$.**

4.Observations : $\mathbf{s} = x_1 - x_0 = (-1{,}6 ; -1{,}6)$ et $\mathbf{y} = g_1 - g_0 = (4{,}8 - 8 ; -4{,}8 - 8) = (-3{,}2 ; -12{,}8)$.

5.Calcul de ρ : $y^T s = (-3{,}2)(-1{,}6) + (-12{,}8)(-1{,}6) = 5{,}12 + 20{,}48 = 25{,}6$, donc **$\rho = 1/25{,}6 = 0{,}0390625$**.

λ

Méthodes de descente : *Méthode quasi-Newton : BFGS*

Exemple: **$f(x, y) = x^2 + xy + 2y^2$,** Pour avoir des chiffres propres, on utilise ici le pas exact $\alpha^* = -(g^T p)/(p^T H p)$, la vraie Hessienne diag(2, 8) n'est utilisée que pour ce calcul du pas, jamais pour la direction.

Itération 1 ($H_0 = I$):

6. Mise à jour de H, en trois temps :
    1. Matrice $s \cdot y^T$ = [[5,12 ; 20,48], [5,12 ; 20,48]], donc $\rho \cdot s \cdot y^T$ = [[0,2 ; 0,8], [0,2 ; 0,8]].
    2. $A = I - \rho \cdot s \cdot y^T$ = [[0,8 ; −0,8], [−0,2 ; 0,2]]. Comme $H_0 = I$ et que $(I - \rho \cdot y \cdot s^T)$ est la transposée de A, le premier terme vaut $A \cdot A^T$ = [[1,28 ; −0,32], [−0,32 ; 0,08]].
    3. Matrice $\rho \cdot s \cdot s^T$ = [[0,1 ; 0,1], [0,1 ; 0,1]]. On additionne : **H_1 = [[1,38 ; −0,22], [−0,22 ; 0,18]]**

7. Vérification (équation sécante) : $H_1 \cdot y$ = (1,38·(−3,2) + (−0,22)(−12,8) ; (−0,22)(−3,2) + 0,18·(−12,8)) = (−4,416 + 2,816 ; 0,704 − 2,304) = **(−1,6 ; −1,6) = s ✓**

λ

Méthodes de descente : *Méthode quasi-Newton : BFGS*

Exemple: **$f(x, y) = x^2 + xy + 2y^2$,** Pour avoir des chiffres propres, on utilise ici le pas exact $\alpha^* = -(g^T p)/(p^T H p)$, la vraie Hessienne diag(2, 8) n'est utilisée que pour ce calcul du pas, jamais pour la direction.

Itération 2 (H_1).

1.Direction : $H_1 \cdot g_1 = (1{,}38 \cdot 4{,}8 + (-0{,}22)(-4{,}8) ; (-0{,}22)(4{,}8) + 0{,}18 \cdot (-4{,}8)) = (6{,}624 + 1{,}056 ; -1{,}056 - 0{,}864) = (7{,}68 ; -1{,}92)$. Donc $\mathbf{p_1 = -H_1 \cdot g_1 = (-7{,}68 ; 1{,}92)}$.

2.Pas exact : $g^T p = 4{,}8 \cdot (-7{,}68) + (-4{,}8)(1{,}92) = -36{,}864 - 9{,}216 = -46{,}08$, et $p^T H p = 2 \cdot 7{,}68^2 + 8 \cdot 1{,}92^2 = 117{,}9648 + 29{,}4912 = 147{,}456$, donc $\alpha = 46{,}08 / 147{,}456 =$ **0,3125**.

3.$x_2 = (2{,}4 - 7{,}68 \cdot 0{,}3125 ; -0{,}6 + 1{,}92 \cdot 0{,}3125) = (2{,}4 - 2{,}4 ; -0{,}6 + 0{,}6) =$ **(0, 0)** ✓

BFGS atteint le minimum en **2 itérations** (soit n = 2 : sur une fonction quadratique en dimension n, BFGS avec pas exact converge en au plus n itérations), contre environ 30 pour la descente de gradient, et sans jamais calculer la Hessienne.

Méthodes de descente

Comparaison

Les méthodes ne diffèrent que par le choix de la direction **p** :

Méthode	Direction p	Idée
Descente de gradient	$\mathbf{p} = -\nabla f(x)$	suivre la plus forte pente
Newton	$\mathbf{p} = -H^{-1}\nabla f(x)$	tenir compte de la courbure
BFGS	$\mathbf{p} = -\tilde{H}\,\nabla f(x)$	comme Newton, mais en devinant la courbure

H est la matrice « Hessienne », expliquée ci-dessous ; $\tilde{H}$ est une approximation de H^{-1}

Optimisation sans contraintes

Taux de convergence

λ

Taux de convergence

Principe. La *vitesse de convergence* mesure à quelle vitesse l'erreur $e_k = \|x_k - x^*\|$ diminue.

Type	Ce qui se passe	Image
Linéaire	l'erreur est multipliée par une constante $\rho < 1$ à chaque itération	on gagne un nombre *constant* de décimales par itération
Superlinéaire	le facteur de réduction lui-même diminue	on gagne de plus en plus de décimales
Quadratique	l'erreur est environ élevée au carré	le nombre de décimales exactes **double**

λ

Taux de convergence

Exemple : combien d'itérations pour atteindre une erreur de 10^{-8} ?

•*Linéaire* avec ρ = 0,5 (erreur divisée par 2), départ à 1 : on veut $0{,}5^k \leq 10^{-8}$, soit $k \geq 26{,}6$, donc **27 itérations**.

•*Quadratique*, départ à 0,1 : les erreurs successives sont 0,1 → 0,01 → 0,0001 → 0,00000001, donc **3 itérations**.

λ

Taux de convergence

Exemple : l'effet du conditionnement κ sur la descente de gradient.

On veut réduire l'écart $f(x) - f^*$ d'un facteur 1000.

Avec le meilleur pas fixe, cet écart est multiplié à chaque itération par $\rho = ((\kappa - 1)/(\kappa + 1))^2$.

Il faut donc k tel que $\rho^k \leq 10^{-3}$, soit $k \geq \ln(10^{-3})/\ln(\rho) = 6{,}91 / |\ln \rho|$.

Fonction	κ	ρ	Itérations (descente de gradient)	Newton
$x^2 + y^2$ (bol rond)	1	0	1	1
$x^2 + 10y^2$	10	$(9/11)^2 \approx 0{,}67$	environ 18	1
$x^2 + 100y^2$ (vallée étroite)	100	$(99/101)^2 \approx 0{,}96$	environ 170	1

Newton résout ces trois cas en une itération car ce sont des fonctions quadratiques

Méthodes de descente

Comparaison

	Descente de gradient	Newton	BFGS
Information nécessaire	gradient	gradient + Hessienne	gradient seul
Coût par itération	faible ($\approx n$)	élevé ($\approx n^3$)	moyen ($\approx n^2$)
Vitesse	linéaire (lente si κ grand)	quadratique (près de x^*)	superlinéaire
Itérations sur le fil rouge	≈ 30	1	2
Quand l'utiliser	n énorme (apprentissage profond, version stochastique)	n petit, Hessienne disponible	n moyen, Hessienne indisponible ou coûteuse

Méthodes de descente

Pièges fréquents.

- Pas fixe trop grand → divergence (voir l'exemple 3 de la descente de gradient).
- Oublier le critère d'arrêt (toujours prévoir max_iter en plus de $\|\nabla f\| < \varepsilon$).
- Gradient mal programmé : vérifier avec (f(x+h) - f(x-h)) / (2*h), la dérivée numérique, avant de lancer l'algorithme.
- Newton lancé trop loin du minimum, ou sur une Hessienne non définie positive (il peut converger vers un maximum ou un point selle).

Fin de la presentation